

9.3 IMU, Linear Velocity and Angular Velocity Calibration

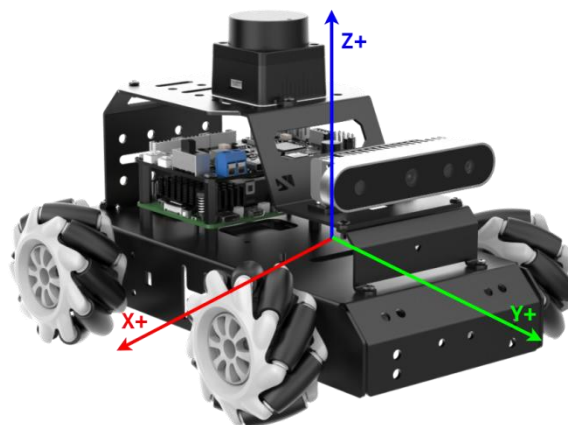
Note:

1. The robot has been calibrated at the factory and does not require additional calibration. The following information is provided for reference only. If you notice significant deviations during movement—such as the robot drifting to one side when moving forward or being unable to travel straight—you may refer to the tutorial below for calibration instructions.
2. Calibration helps reduce deviations, but inherent hardware variations may still occur. Adjust the calibration to a level that meets your specific needs and requirements.

If the robot exhibits deviations during operation, it may require calibration of the IMU, linear velocity, and angular velocity. Once the calibration is complete, the robot can resume normal operation.

1. IMU Calibration

The IMU (Inertial Measurement Unit) is a device used to measure the three-axis attitude angles (angular velocity) and acceleration of an object. The IMU consists of a gyroscope and an accelerometer, providing 6 degrees of freedom to measure angular velocity and acceleration in three-dimensional space.



Upon receiving the first IMU message, the system will prompt you to maintain

the IMU in a specific orientation. After this, press Enter to record the measurement values. Once measurements are completed in all 6 directions, the system will calculate the calibration parameters and save them to the specified YAML file. The specific steps are as follows:

Note: The input commands are case-sensitive, and keywords can be autocompleted by pressing the Tab key.

1) Start the robot, and access the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

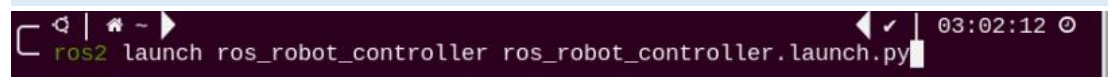
3) Execute the command to disable the app auto-start service.

```
~/stop_ros.sh
```



4) Enter the command and hit Enter key to enable the chassis control node:

```
ros2 launch ros_robot_controller ros_robot_controller.launch.py
```



5) Open a new ROS2 command-line terminal, then input the following command, and hit Enter key to initiate IMU calibration:

```
ros2 run imu_calib do_calib --ros-args -r
```

```
imu:=/ros_robot_controller/imu_raw --param
```

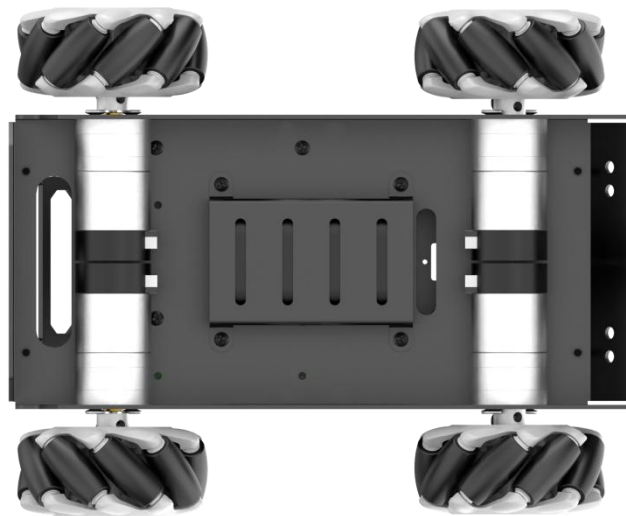
```
output_file:=/home/ubuntu/ros2_ws/src/calibration/config/imu_calib.yaml
```

```
|
```



6) When the following prompt appears, position the robot in the first-person perspective, aligning it to face forward, and then press Enter. The initial direction it faces will be defined as "forward," and all subsequent placements should be based on this reference.

```
Orient IMU with X+ axis up and press Enter
```



After you have successfully calibrated in each direction, the following prompt will appear.

```
Recording measurements... Done.
```

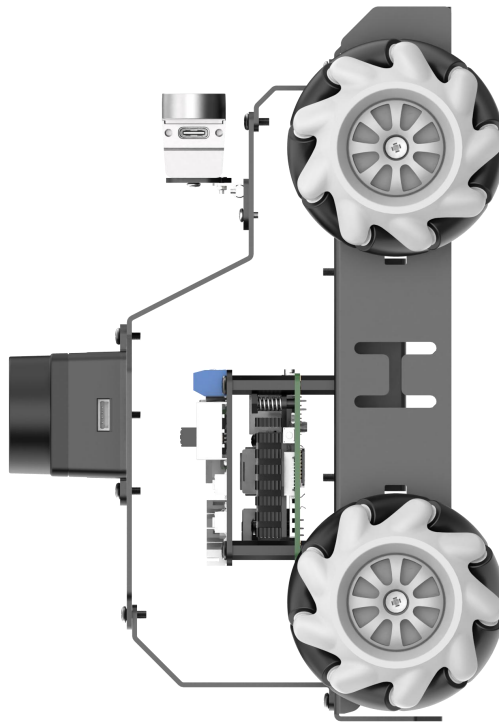
7) Align the robot to the rear, then press Enter.

```
Orient IMU with X- axis up and press Enter
```



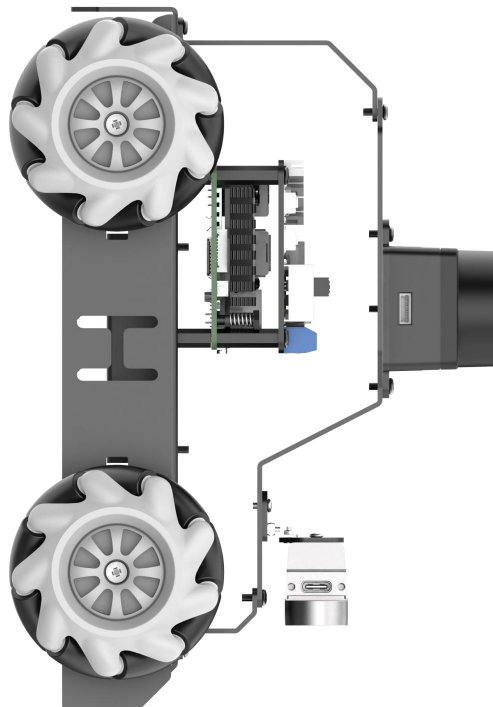
8) Align the robot to the left, then hit Enter.

```
Orient IMU with Y+ axis up and press Enter
```



9) Align the robot to the right, then press Enter.

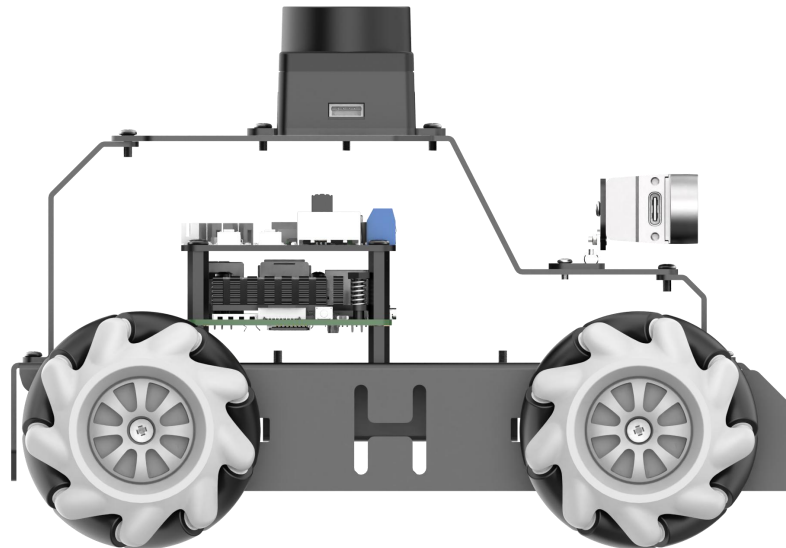
```
Orient IMU with Y- axis up and press Enter
```



10) Lift the robot and position it facing upwards, then press Enter. When placing it vertically, ensure stability to avoid any tipping or collisions. Use your hand for support to protect the depth camera and the 2D pan-tilt from potential

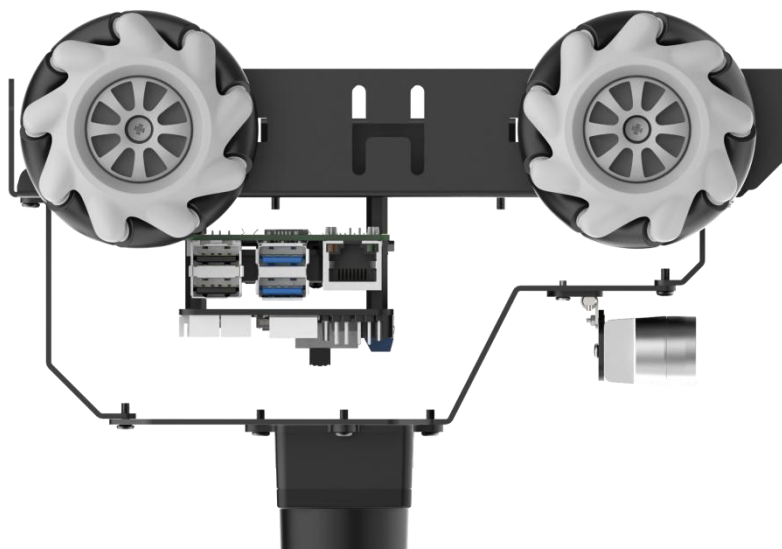
damage.

```
Orient IMU with Z+ axis up and press Enter
```



11) Place the robot as pictured, then hit Enter.

```
Orient IMU with Z- axis up and press Enter
```



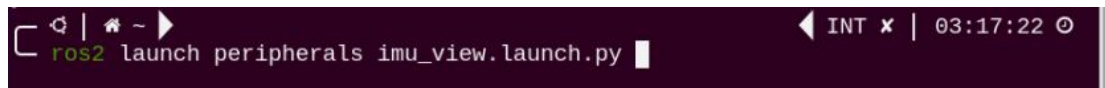
12) If the below prompt shows up, it means the calibration is complete. To exit,

use short-cut “ctrl+c”.

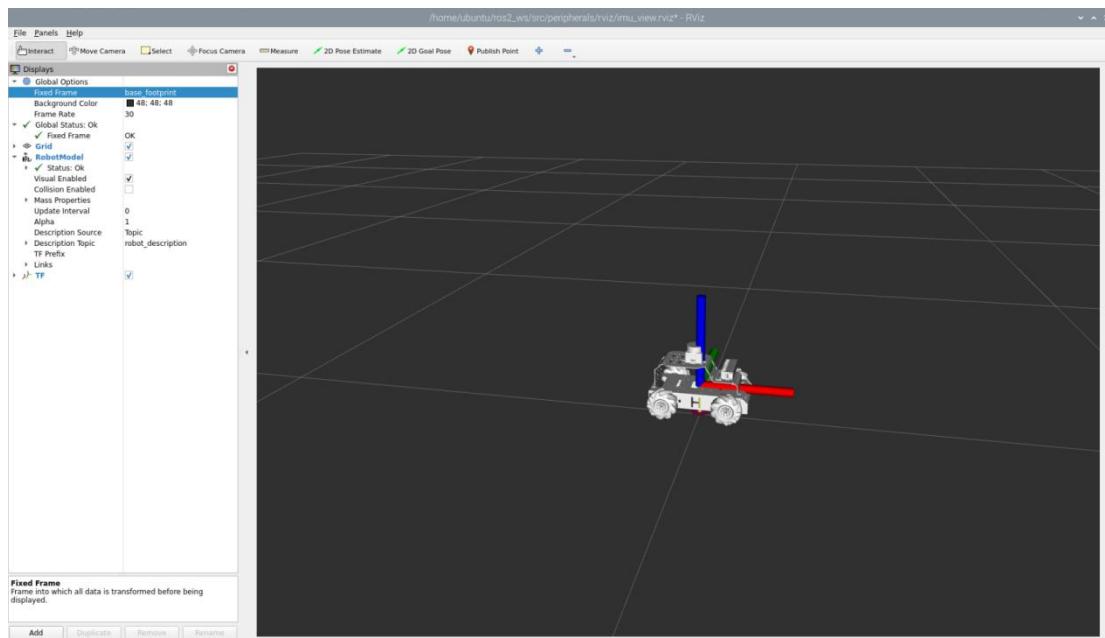
```
Computing calibration parameters... Success!  
Saving calibration file... Success!
```

13) After calibration, execute the command to verify the calibrated model.

```
ros2 launch peripherals imu_view.launch.py
```



14) You can move the robot to see if it matches the angle and direction of the model.



15) To exit the game, press Ctrl+C in the terminal.

After exiting, you can re-enable the app service either by entering a command or simply restarting the robot. Without this service running, app-related functions will not work. If the robot is restarted, the app service will start automatically.

To manually start the app service:

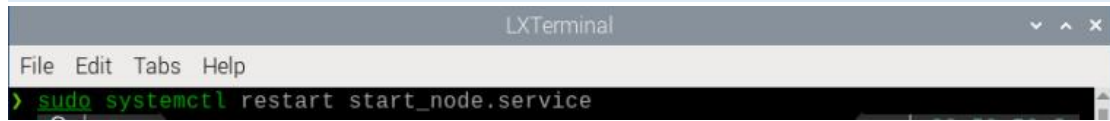


Click  and enter the command, and press **Enter**. Wait for the buzzer to beep, indicating the service has started.

Note: Be sure to run the command in the **system terminal**, not inside the

Docker container.

```
sudo systemctl restart start_node.service
```



2. Angular Velocity Calibration

The Ackermann version does not require angular velocity calibration.

For robots that do need angular velocity calibration, the process involves the robot performing a full, independent rotation. During this test, it's essential to mark the robot's starting orientation to accurately observe any drift or deviation throughout the rotation.

The detailed calibration steps are as follows:

Note: Command input is case-sensitive. You can use the Tab key to auto-complete keywords.

- 1) Position the robot on a flat surface, and place a piece of tape or another marker directly in front of its center to serve as a reference point.
- 2) Start the robot, and connect it to the robot system desktop using VNC.
- 3) Click-on  to open the command-line terminal.
- 4) Enter the command and hit Enter to disable the app auto-start service.

```
~/stop_ros.sh
```



- 5) Before calibration, access the directory of calibration configuration files to open the files.

```
cd ~/ros2_ws/src/driver/controller/config && vim calibrate_params.yaml
```



6) Modify the angular velocity parameter to 1.0, then proceed with the calibration.

```
1 /**:
2   ros_parameters:
3     linear_correction_factor_tank: 1.0
4     linear_correction_factor: 1.0
5     angular_correction_factor: 1.0
```

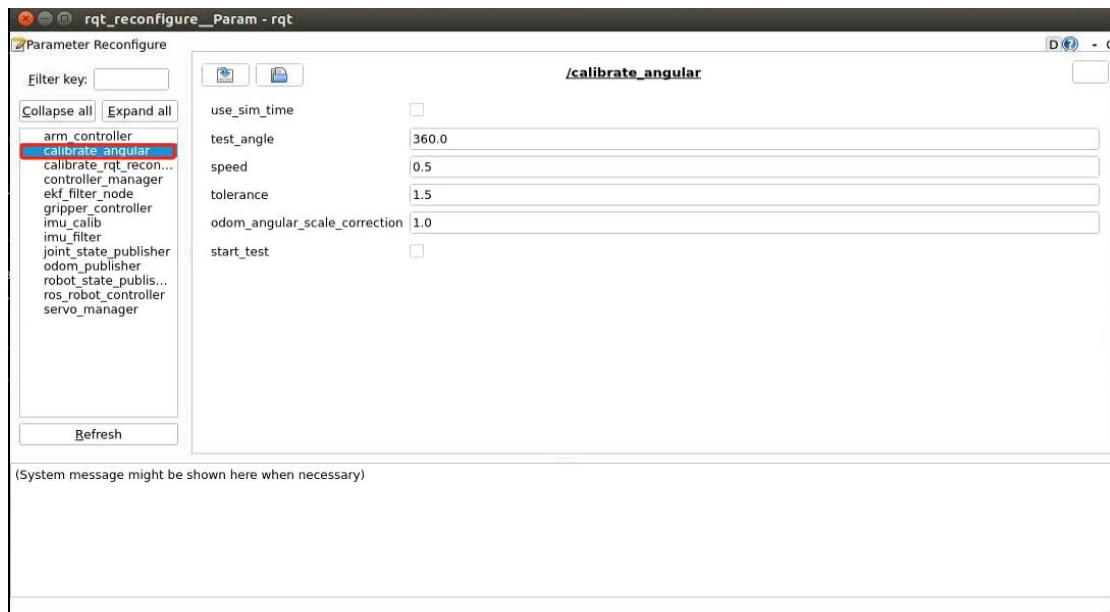
7) After modifying, press "ESC" . Enter ":\wq" to exit and save.

8) Enter the command and press enter to start the angular velocity calibration.

ros2 launch calibration angular_calib.launch.py

```
ros2 launch calibration angular_calib.launch.py
```

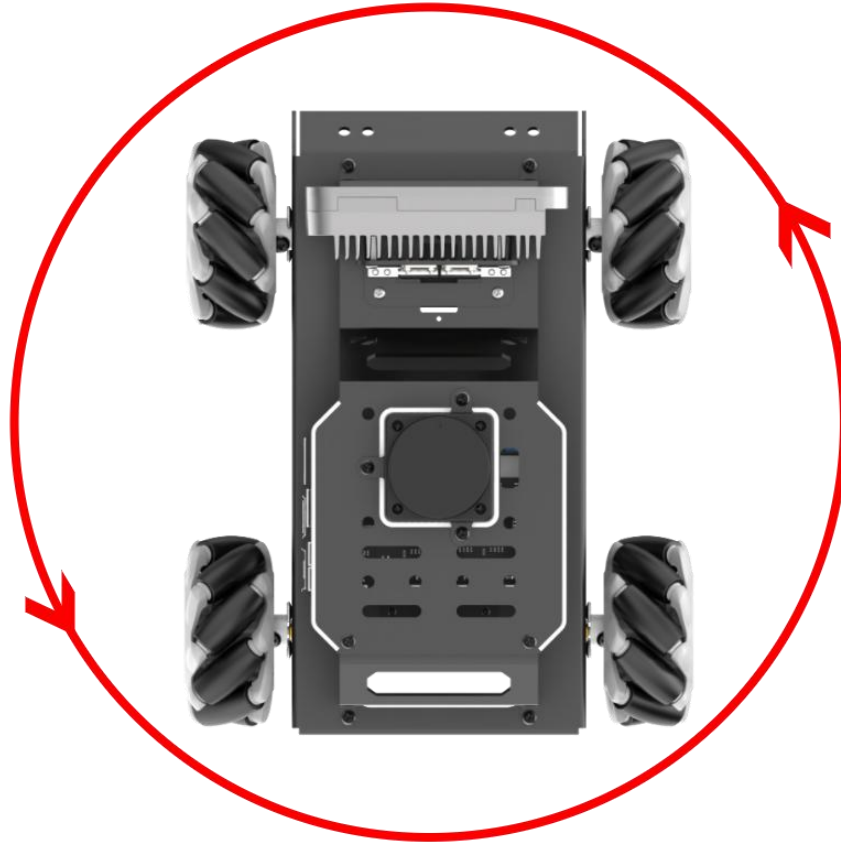
9) Click "calibrate_angular" on the left side. The calibration interface will appear as shown below.



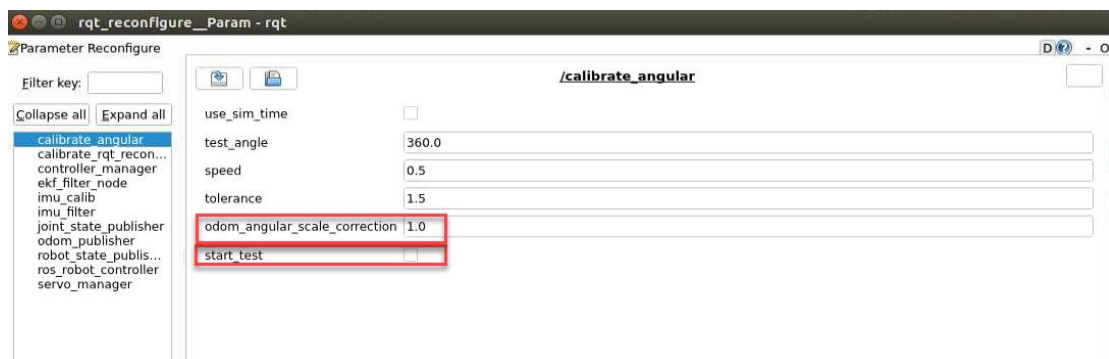
- test_angle: Specifies the test rotation angle, with a default value of 360°.
- speed: Indicates the linear velocity, set by default to 0.15 meters per second.
- tolerance: Represents the allowable error margin. A smaller value leads to more noticeable oscillation when the robot reaches the target position.
- odom_angle_scale_correction: Used to adjust the scale correction for

odometry angle measurements.

- **start_test**: A button to initiate the test for odometry angle scale correction.



Ensure the robot is properly aligned, with the marker positioned directly in front of it. Then check the "**start_test**" option to begin the test—this will cause the robot to rotate in place. If the robot does not complete a full rotation, adjust the "**odom_angle_scale_correction**" parameter, which controls the scaling of the motor's rotational output. It is recommended to fine-tune this value in increments of 0.01.



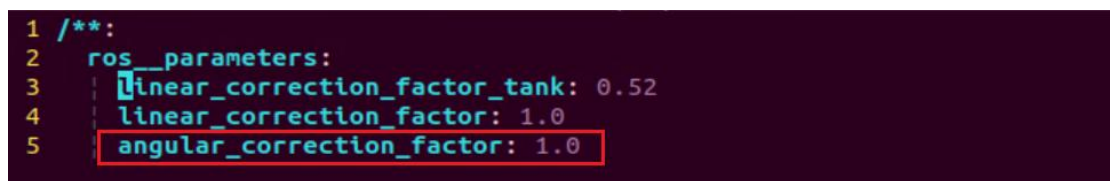
10) After calibration, enter the command to navigate to the directory containing

calibration configuration files. Modify to the calibrated parameter value.

```
cd ~/ros2_ws/src/driver/controller/config && vim calibrate_params.yaml
```



11) Press 'i' key to navigate to the editing mode, and modify the value of "angular_correction_factor" to the adjusted value of "odom_angle_scale_correction".



Note: The aforementioned operations are conducted on the Mecanum-wheel version.

12) After modification, press the "ESC" key, enter ":wq" to exit and save the changes.

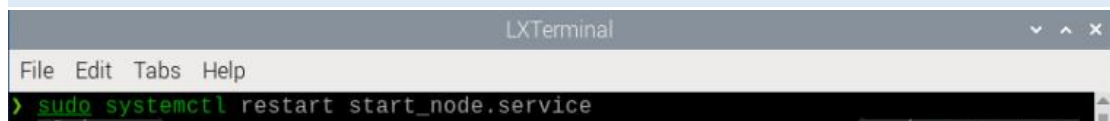
13) If you want to exit the game, press "Ctrl+C" in the terminal interface.

14) After finishing the game, you can enable the app service either by entering a command or by restarting the robot. If the app service is not enabled, related app functions will not operate. Restarting the robot will automatically enable the app.

Click  and enter the corresponding command, press Enter to launch the app, and wait for the buzzer to beep as confirmation.

Note: Be sure to enter the command in the system path, not within the Docker container.

```
sudo systemctl restart start_node.service
```



3. Linear Velocity Calibration

The Ackermann version does not require angular velocity calibration.

Note: The input command is case-sensitive, and keywords can be completed using the Tab key.

Position the robot on a flat and open surface. Mark the starting point with tape or any other indicator in front of the robot, and position the endpoint tape or another marker 1 meter ahead of the robot.

1) Start the robot, and access the robot system desktop using VNC.

- 2) Click-on  to open the command-line terminal.
- 3) Execute the command and hit Enter to disable the app auto-start service.

```
~/ .stop_ros.sh
```



4) Before calibration, navigate to the directory of calibration configuration files to open them.

```
cd ~/ros2_ws/src/driver/controller/config && vim calibrate_params.yaml
```



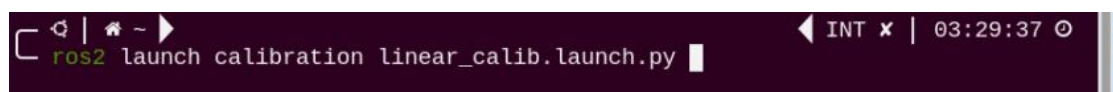
5) Modify the linear velocity parameter "linear_correction_factor" to 1.0, then proceed with the calibration.



6) After modifying, press "ESC" . Enter ":wq" to exit and save.

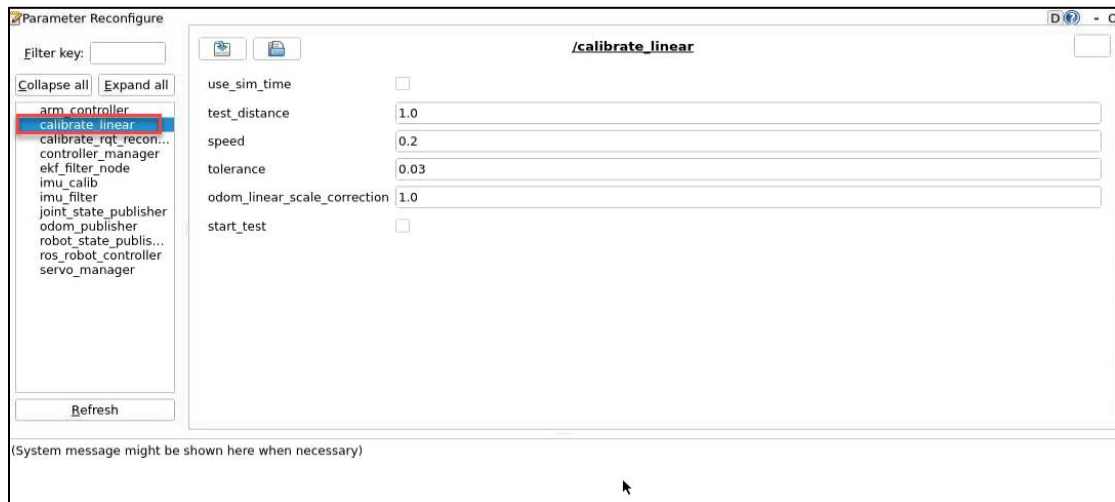
7) Enter the command and press enter to start the linear velocity calibration.

```
ros2 launch calibration linear_calib.launch.py
```



8) Click "calibrate_linear" on the left side. The calibration interface will appear

as shown below.



The meanings of parameters on the left side of the interface are as follows:

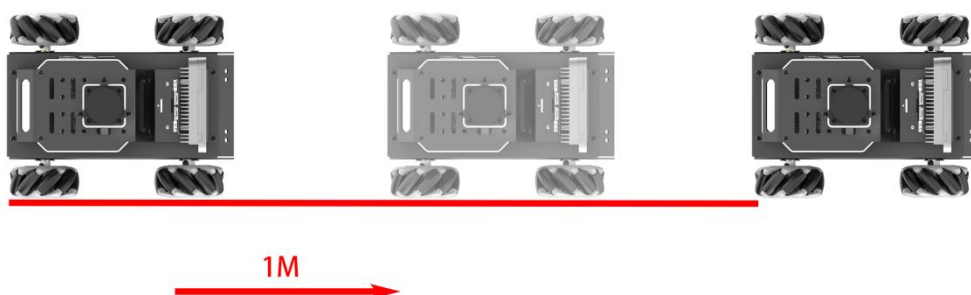
The first parameter "test_distance" represents the testing distance, with a default value of 1 meter.

The second parameter "speed" represents the linear velocity, with a default value of 0.2 meters per second.

The third parameter "tolerance" represents the error value. A smaller error value results in greater robot shaking after reaching the target position.

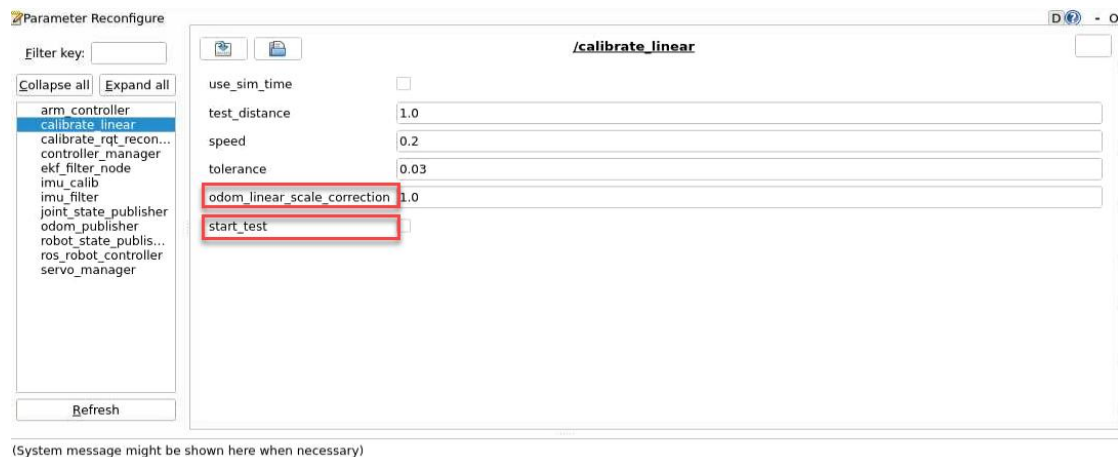
The fourth parameter "odom_linear_scale_correction" represents the odometer linear scale correction.

The fifth parameter "start_test" is the button to start testing the odometer linear scale correction.



9) Ensure the robot is properly aligned and positioned at the starting point marker. Check the box for "start_test" and the robot will move forward.

Observe if the robot travels in a straight line. If there is deviation, adjust the value of "odom_linear_scale_correction". This value adjusts the motor's scaling factor for forward movement. It is recommended to adjust this value by increments of 0.01 each time.



10) After calibration, enter the command to navigate to the directory containing calibration configuration files. Modify to the calibrated parameter value.

```
cd ~/ros2_ws/src/driver/controller/config && vim calibrate_params.yaml
```



11) Press 'l' key to navigate to the editing mode, and modify the value of "linear_correction_factor" to the adjusted value of "odom_linear_scale_correction".



12) After modification, press the "ESC" key, enter ":wq" to exit and save the changes.

13) If you want to exit the game, press "Ctrl+C" in the terminal interface.

After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically

enabled.

Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```

